MINISTRY OF EDUCATION AND TRAINING
NATIONAL ACADEMIC RESEARCH INSTITUTE
FACULTY OF INFORMATION TECHNOLOGY AND CYBERSECURITY

***

ACADEMIC CAPSTONE RESEARCH REPORT
CRYPTOGRAPHY AND INFORMATION ASSURANCE

REAL-WORLD APPLICATIONS OF RSA: CRYPTOGRAPHIC FOUNDATIONS, PRACTICAL IMPLEMENTATION, AND OFF-CHAIN BLOCKCHAIN AUTHENTICATION

RESEARCH PROJECT MEMBER LIST / STUDENT IDENTIFICATION

No.	Full Name	Student ID
1	.....	.....
2	.....	.....
3	.....	.....
4	.....	.....
5	.....	.....
6	.....	.....
7	.....	.....

HANOI, 2026

Real-World Applications of RSA: Cryptographic Foundations, Practical Implementation, and Off-Chain Blockchain Authentication

Academic Research Paper in Cryptography, Cybersecurity, and Blockchain Systems

Abstract— The RSA algorithm remains a foundational pillar of modern public-key cryptography. While public blockchain consensus protocols predominantly utilize elliptic-curve cryptography (ECC) for on-chain state compactness, RSA plays an enduring role in enterprise infrastructure, Public Key Infrastructure (PKI), and internal gateway authorization. This report provides an academic investigation of real-world RSA applications across theory, operational justification, and systems engineering. We formulate RSA's mathematical foundations, contrasting RSA-OAEP key transport with RSA-PSS digital signatures (WHAT). We analyze why RSA is favored in specific trust architectures, providing an evidence-based comparison against ECDSA and Schnorr signatures (WHY). Finally, we design, implement, and benchmark an educational off-chain cryptocurrency withdrawal authentication system using RSA-PSS and SHA-256 (HOW). Empirical benchmarks across RSA (2048, 3072, 4096 bits) and ECDSA (secp256k1) on identical hardware demonstrate RSA's distinct operational profile: while RSA signing is slower than ECDSA, RSA public exponent verification (e = 65,537) executes significantly faster across all tested key sizes—RSA-2048 is 11.32x faster, RSA-3072 is 6.18x faster, and RSA-4096 is 4.17x faster than ECDSA verification. At the approximately equivalent 128-bit classical security level (RSA-3072 vs. ECDSA-secp256k1), RSA-3072 verification is 6.18x faster than ECDSA, whereas ECDSA signing is 5.82x faster than RSA-3072. Adversarial security scenario testing demonstrates that combining cryptographic digital signatures with application-level timestamp freshness and nonce replay caches significantly strengthens request authentication against payload tampering and replay attacks, although complete system security requires defense-in-depth across transport encryption, secure key custody, and operational controls. Evaluation against the bundled RSA-PSS 2048-bit SHA-256/MGF1-SHA256 Wycheproof test-vector set (version 0.9 snapshot, 108 cases) confirms that the verification engine correctly accepts all 63 expected-valid test cases and rejects all 45 expected-invalid test cases (0 skipped, 0 errors), validating execution against this selected verification suite.

Keywords: *RSA Cryptography, RSA-PSS, Digital Signatures, PKI, Hybrid Encryption, Cryptocurrency Security, Off-chain Authentication, Empirical Evaluation.*

1. Introduction and Research Scope

First documented by Ron Rivest, Adi Shamir, and Leonard Adleman in their April 1977 MIT research memorandum and published in Communications of the ACM in 1978 [1], and formally standardized under PKCS #1 / RFC 8017 [2], the RSA algorithm solved the historic key distribution problem by separating the private transformation used for decryption and signing from the public transformation used for encryption and verification. For nearly five decades, RSA has underpinned digital commerce, software distribution, and enterprise identity management.

In modern blockchain environments, significant architectural debate centers on algorithm selection. Major public networks—including Bitcoin and Ethereum—rely on Elliptic Curve Cryptography (ECC) for native on-chain transaction signatures [8], [12], [13]. In enterprise cryptocurrency operations, transaction handling does not occur solely on-chain; systems may employ layered off-chain microservices, secure hardware enclaves or Hardware Security Modules (HSMs), and Transport Layer Security (TLS 1.3) tunnels [3] to authenticate and authorize requests before dispatching them to blockchain nodes. This report investigates real-world RSA applications across three dimensions: (1) WHAT: mathematical principles and operational distinctions between RSA-OAEP key transport and RSA-PSS signatures; (2) WHY: security justification of RSA and an evidence-based comparison with ECDSA and Schnorr; (3) HOW: practical design, implementation, and empirical evaluation of an educational off-chain cryptocurrency withdrawal authentication system using RSA-PSS.

2. RSA Fundamentals and Real-World Applications

2.1 Mathematical Foundations and Operating Mechanics

The security of RSA rests on the integer factorization problem—the computational difficulty of determining prime factors p and q from modulus n = p · q [1], [2]. The operational lifecycle comprises: (i) computing Euler's totient φ(n) = (p − 1)(q − 1); (ii) selecting public exponent e coprime to φ(n) (standardized as Fermat prime e = 65,537 = 2^{16} + 1); and (iii) deriving private exponent d ≡ e^{-1} (mod φ(n)). The public key comprises (e, n), and the private key comprises (d, n) or Chinese Remainder Theorem (CRT) parameters (p, q, dP, dQ, qInv). By Euler's theorem, for any message m coprime to n, m^(e·d) ≡ m (mod n). Textbook RSA operates directly on integers (c = m^e mod n; m = c^d mod n), but is vulnerable to algebraic homomorphic manipulation and chosen-ciphertext attacks. Secure real-world implementations mandate modern randomized padding schemes.

2.2 Distinction Between RSA Encryption and RSA Digital Signatures

RSA supports two distinct operational primitives designed for contrasting security goals:

• Asymmetric Encryption (Confidentiality): The sender encrypts plaintext using recipient public key (e, n); only private key d can decrypt. Modern systems employ RSA-OAEP (RFC 8017 Section 7.1 [2]). In the Random Oracle Model (ROM), Bellare and Rogaway (1994) [17] introduced OAEP and established semantic security against chosen-plaintext attacks (IND-CPA) and plaintext awareness (implying IND-CCA1 security for general trapdoor permutations). Subsequently, Shoup (2001) [15] demonstrated that the general IND-CCA2 security proof claim could not be established for arbitrary trapdoor permutations via standard black-box reductions under standard one-wayness, and proposed the provably secure variant OAEP+. Fujisaki, Okamoto, Pointcheval, and Stern (2001) [16] resolved the security of RSA-OAEP, introducing the concept of partial-domain one-wayness to prove that OAEP achieves IND-CCA2 security in the ROM when the permutation is partial-domain one-way, and showing that for the RSA permutation, partial-domain one-wayness is mathematically equivalent to full-domain one-wayness (the standard RSA assumption), though the concrete reduction is non-tight for practical RSA key lengths. Because RSAES-OAEP payload size is bounded by mLen <= k - 2hLen - 2 (RFC 8017 Section 7.1.1; 190 bytes for RSA-2048 with SHA-256), industrial architectures deploy it within hybrid encryption for RSA key transport—encapsulating ephemeral symmetric keys (e.g., 256-bit AES-GCM per NIST SP 800-56B Rev. 2 [5]) to secure high-throughput bulk data streaming.

• Digital Signatures (Authenticity, Integrity, Non-Repudiation Evidence): Under RFC 8017 Section 8.1 / 9.1 and NIST FIPS 186-5 [2], [4], RSA-PSS is the state-of-the-art standard. Given original message octet string M, the signing sequence computes message digest mHash = Hash(M), generates encoded message octet string EM = EMSA-PSS-ENCODE(M, emBits = modBits − 1) incorporating a randomized salt, converts octets to integer message representative m = OS2IP(EM), computes signature representative s = RSASP1(SK, m) (via private exponent d or CRT parameters), and formats signature octet string S = I2OSP(s, k).

The verifier parses integer s = OS2IP(S), applies public modular exponentiation to recover integer representative m = RSAVP1(PK, s) = s^e mod n, converts integer to octet string EM = I2OSP(m, emLen), and executes EMSA-PSS-VERIFY(M, EM, emBits). Crucially, modular exponentiation (RSAVP1) alone does not establish signature validity; EMSA-PSS decoding—unmasking the randomized salt via MGF1, checking padding bytes, and confirming hash consistency against M—is indispensable. PSS achieves a tight security reduction to the RSA problem in the ROM (EUF-CMA). Cryptographic signatures furnish non-repudiation evidence, while full non-repudiation requires identity binding, secure custody, and operational controls.

• Cryptographic Key Separation (NIST FIPS 186-5 Section 5.1 / SP 800-57): A vital security mandate in asymmetric cryptography is strict key separation. NIST FIPS 186-5 Section 5.1 explicitly dictates that an RSA key pair designated for digital signatures shall not be used for any other purpose, such as key establishment or encryption (RSA-OAEP). Multi-purpose key reuse exposes systems to cross-protocol interaction attacks, undermines non-repudiation auditing, and expands compromise blast radius. Architectures deploying both signing and key transport must enforce separate key pairs, distinct certificate bindings, and independent key-management policies.

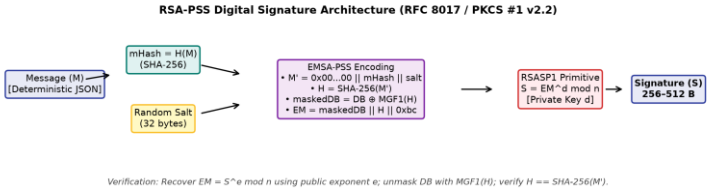


Figure 1. Operational workflow of the RSA-PSS probabilistic digital signature scheme (RFC 8017 / FIPS 186-5).

2.3 Real-World Application Breadth

In industrial systems, RSA is deployed across three primary domains: (1) Hybrid Encryption and RSA Key Transport: RSA is not used to encrypt bulk data directly due to algebraic length constraints: under RFC 8017 Section 7.1.1 [2], plaintext size for RSA-OAEP is bounded by mLen <= k - 2hLen - 2 (190 bytes for RSA-2048 with SHA-256). Instead, RSA-OAEP is deployed for RSA key transport—encapsulating ephemeral symmetric keys (e.g., 256-bit AES-GCM per NIST SP 800-56B Rev. 2 [5])—combining asymmetric key establishment with high-throughput symmetric streaming. (2) Code Signing and Firmware Integrity: Operating systems, application marketplaces, and secure bootloaders frequently deploy RSA digital signatures (including RSA-PSS and PKCS#1 v1.5) to verify executable integrity before launch. (3) Digital Certificates and PKI: In TLS 1.3 (RFC 9846 [3], superseding RFC 8446), static RSA key exchange was entirely removed in favor of ephemeral Diffie-Hellman ((EC)DHE) and PSK-DHE (psk_dhe_ke) to guarantee forward secrecy. In contrast, PSK-only mode (psk_ke) does not provide forward secrecy against compromised pre-shared keys, and 0-RTT early data lacks forward secrecy and is vulnerable to replay attacks. Post-quantum forward secrecy additionally requires negotiating a supported hybrid mechanism (e.g., RFC 10024). Thus, RSA's modern TLS 1.3 role is strictly endpoint authentication: RSA X.509 certificates authenticate servers via RSA-PSS CertificateVerify messages (RFC 9846), while RFC 9963 provides a restricted legacy exception for client authentication with PKCS#1 v1.5 in constrained environments.

3. Why RSA? Security Purpose and Comparative Evaluation

RSA addresses the core security objectives of information assurance: confidentiality (via hybrid key transport), entity authentication, data integrity, and non-repudiation evidence. Table 1 compares RSA with elliptic curve alternatives: ECDSA (secp256k1) [12], [13], Schnorr (BIP 340) [8], and Ed25519 [11], with security strengths benchmarked according to NIST SP 800-56B Rev. 2 (Table 2 and Appendix D) [5] and generic elliptic-curve strength mappings from NIST SP 800-57 Part 1 Rev. 5 [6].

Evaluation Metric	RSA-2048	RSA-3072	ECDSA (secp256k1)	Schnorr (BIP 340)	Ed25519 (RFC 8032)
Hard Problem	Factoring	Factoring	EC Discrete Log	EC Discrete Log	Edwards EC-DLOG
Security Strength	112 bits [5]	128 bits [5]	~128 bits (generic EC mapping [6])	128 bits (generic EC mapping [6])	128 bits (generic EC mapping [6])
Key Size / Sig Size	Modulus: 256 B Sig: 256 B DER PubKey: ~294 B	Modulus: 384 B Sig: 384 B DER PubKey: ~422 B	PubKey: 33 B (comp) Bitcoin: 70–72 B (71 B sample) Ethereum: 65 B (r‖s‖v), RLP; 64 B (ERC-2098)	PubKey: 32 B (x-only) BIP341: 64 B (DEFAULT) or 65 B (explicit sighash)	PubKey: 32 B Sig: 64 B [11]
Relative Verify Speed	Empirical: 0.021 ms (11.32x vs ECDSA)	Empirical: 0.039 ms (6.18x vs ECDSA)	Empirical: 0.242 ms (Baseline: 1.0x)	Batch verify supported; not benchmarked [8]	High efficiency in lit. [11]; not benchmarked

Table 1. Systematic comparison of asymmetric cryptographic algorithms across security and operational metrics.

3.1 Storage and Verification Trade-Offs in Blockchain Consensus Layers

A central requirement of public blockchains is minimizing ledger bloat and bandwidth overhead. In Bitcoin and Ethereum, transaction fees and execution costs penalize excessive data footprints. Under Bitcoin's economic model, fees equal transaction virtual size (vSize in vB) multiplied by prevailing feerate (sat/vB), whereas Ethereum costs are measured in gas for execution steps and calldata serialization [12], [13]. In both networks, compact signatures and public keys are paramount for throughput and decentralized state validation. To evaluate transaction overhead accurately under Bitcoin BIP 141 (Segregated Witness) [9], transaction weight is formulated as Weight = (Base Size * 3) + Total Size, and virtual size is vSize = ceil(Weight / 4). A standard 1-input, 2-output SegWit P2WPKH transaction has a baseline virtual size of ~141 vB (~210–220 serialized bytes). For a typical P2WPKH witness stack—comprising 1 byte for stack item count, 1 byte for signature length prefix, ~70–72 bytes for the DER signature plus sighash byte, 1 byte for public key length prefix, and 33 bytes for the compressed public key—the witness totals approximately 107–109 bytes [12]. In a hypothetical architecture where RSA-2048 were used for native transaction authorization, explicitly assuming two separate stack items (a 256-byte RSA modulus and a 256-byte RSA signature, each prefixed by a 3-byte CompactSize length prefix for 256-byte items), the witness size equals 1 + 3 + 256 + 3 + 256 = 519 bytes. Compared with standard P2WPKH, this adds approximately 410–412 witness bytes per input. Under BIP 141 weight rules, this witness overhead adds 410–

412 weight units, which corresponds to approximately 103 additional virtual bytes (~103 vB per input, subject to transaction weight rounding). For a representative baseline 141-vB transaction, this hypothetical expansion is approximately 73% (expanding to ~244 vB), proportionally increasing miner fees. Crucially, this analysis models a hypothetical transaction format; native Bitcoin RSA verification is not supported (Bitcoin script lacks native RSA verification opcodes, requiring consensus hard or soft forks), and the 73% expansion assumes both RSA values are carried in each input. Alternative key-storage designs—such as registering public keys in an on-chain key directory, smart-contract registry, or referencing them via script hashes—would substantially alter and reduce this overhead. Similarly, native Ethereum Externally Owned Account (EOA) transaction authorization mandates ECDSA over secp256k1 [13]. In network-level transaction envelopes (legacy, EIP-2930, and EIP-1559), Ethereum serializes signatures as discrete Recursive Length Prefix (RLP) fields encoding r, s, and v (or yParity for typed envelopes). While off-chain systems commonly concatenate these into a 65-byte r||s||v string, ERC-2098 also defines a compact 64-byte representation that packs yParity into the upper bit of s. While Ethereum smart contracts can execute raw modular exponentiation via the EVM precompile at address 0x05 (EIP-198), this precompile computes B^E mod M only; complete RSA-PSS verification additionally requires separate contract bytecode to execute MGF1 mask generation, extract the encoded message, and validate hash/padding integrity. For native transaction envelopes, consensus rules strictly require ECC. In contrast, BIP 340 Schnorr signatures over secp256k1 integrated via BIP 341 (Taproot) are exactly 64 bytes under SIGHASH_DEFAULT (or 65 bytes when an explicit non-default sighash byte is appended), paired with 32-byte x-only public keys, and enable two-round multi-signature aggregation via MuSig2 (BIP 327 [10]), allowing n-of-n signers to publish a single compact 64-byte signature on-chain—a capability unattainable with standard RSA.

3.2 The Operational Value of RSA in Off-Chain Systems

Despite on-chain limitations, RSA provides distinct benefits in off-chain exchange backends: (1) Asymmetric Verification Speed: RSA public exponentiation with $e = 65{,}537 = 2^{16} + 1 = 10000000000000001_2$ (17 bits, Hamming weight 2) requires only 16 modular squarings and 1 modular multiplication under straightforward square-and-multiply exponentiation (17 operations total), enabling rapid verification at edge gateways. (2) Hardware Security Module (HSM) Integration: Enterprise banking infrastructure and secure enclaves feature mature hardware support for RSA modular arithmetic. (3) Avoidance of Scalar Nonce Reuse Hazards: Standard ECDSA is fragile against nonce failure: if the signing nonce k repeats or exhibits slight bit bias, the private key is algebraically exposed. While RSA-PSS eliminates scalar nonce leakage, it does not remove all side-channel risks: implementations must employ base blinding (randomizing the base integer with a secret factor r: $x' = x \cdot r^e \bmod n$) to mitigate timing side channels. Crucially, blinding alone does not prevent hardware fault injection such as the Bellcore attack on Chinese Remainder Theorem (CRT) implementations (Boneh et al. [7]). In CRT signing, the signer computes $s_p = m^{d_p} \bmod p$ and $s_q = m^{d_q} \bmod q$, where $m = \text{OS2IP}(EM)$ is the integer message representative from EMSA-PSS-ENCODE(M) (RFC 8017 Section 8.1 / 9.1 [2]), satisfying $s^e \equiv m \pmod n$. Bellcore fault attacks operate under two distinct formulations:

• Two-Signature Formulation: Requires one correct signature s and one faulty signature s' generated from the exact same representative $m = \text{OS2IP}(EM)$. If a fault corrupts s_q into s'_q while s_p is uncorrupted, s' satisfies $s' \equiv s \pmod p$ and $s' \not\equiv s \pmod q$, yielding $\gcd(s - s', n) = p$ (or q if s_p is faulted); thus $\gcd(s - s', n) \in \{p, q\}$. Crucially, because RSA-PSS randomizes the salt, independent signing runs on message M produce differing encodings ($EM_1 \neq EM_2$) and representatives ($m_1 \neq m_2$); two signatures with differing salts do not satisfy this condition.

• Single-Fault Formulation: An attacker knowing the exact representative m of faulty signature s' factors n via $\gcd((s')^e - m, n) \in \{p, q\}$. This assumes knowledge of the exact encoded representative m of that specific execution, not merely application message M. While any valid signature reveals its own representative via $s^e \bmod n$, it does not reveal the representative of an independently salted faulty execution.

Countermeasures must remain strictly differentiated: (i) Base blinding mitigates timing side channels, NOT CRT faults; (ii) CRT fault defense requires post-computation verification—verifying $m_{check} = (s^e \bmod n)$ against $m = \text{OS2IP}(EM)$ and checking EMSA-PSS consistency before releasing s—or (iii) redundant fault-detection hardware.

4. Proposed System Design and Implementation

We design and evaluate an educational, off-chain architecture: Using RSA-PSS Digital Signatures to Authenticate Off-Chain Withdrawal Requests in Cryptocurrency Exchanges. Important Technical Boundary: The proposed educational architecture operates strictly off-chain within an internal authorization boundary to authenticate withdrawal requests before they reach hot-wallet signers; it does not replace on-chain ECDSA/Schnorr consensus signatures on Bitcoin or Ethereum, nor does it represent the proprietary architecture of any specific commercial exchange.

4.1 Architecture and Component Decomposition

As illustrated in Figure 2, the authentication pipeline decouples request origination from on-chain broadcast across four components:

1. Withdrawal Service (Client/Frontend): Formats the withdrawal request: blockchain network, recipient address, withdrawal amount (in exact integer atomic units, e.g., wei), a UUIDv4 nonce, and a UNIX epoch timestamp.
2. RSA Signing Component: Encapsulated within a secure enclave or signing service. It formats the payload using deterministic JSON serialization, computes the SHA-256 digest, and signs via RSA-PSS using the exchange private key.
3. Verification Service: An edge gateway that extracts the exchange RSA public key, reconstructs the message digest, and validates the PSS signature.
4. Request Validation Engine: Enforces stateful application rules: evaluates timestamp freshness (±300s skew) and checks nonce uniqueness against a replay cache.

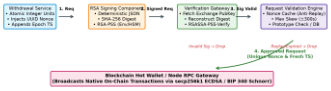


Figure 2. Proposed off-chain cryptocurrency withdrawal authentication architecture using RSA-PSS.

4.2 Deterministic JSON Serialization and Pipeline Algorithm

Cryptographic signature verification fails if distributed microservices serialize JSON keys or whitespace inconsistently. Our system implements deterministic JSON serialization (sorting dictionary keys lexicographically, eliminating whitespace, and encoding to UTF-8). Note: While inspired by canonical formatting, this custom serialization does not implement full RFC 8785 (which requires ECMAScript number formatting and UTF-16 code unit ordering); it provides deterministic byte-level consistency for the defined data schema. Per RFC 8017 Section 9.1, EMSA-PSS encoding specifies emBits = modBits − 1 to ensure that the encoded message representative m = OS2IP(EM) satisfies m < n. For a 2048-bit modulus (modBits = 2048), emBits = 2047 bits and emLen = ceil(emBits / 8) = 256 bytes, with salt length constrained by sLen <= emLen − hLen − 2 (for SHA-256 hLen = 32, sLen <= 222 bytes; our pipeline deploys sLen = 32 bytes). Algorithm 1 details the complete lifecycle.

```
Algorithm 1: Off-Chain Withdrawal Authentication Pipeline
Input:  Request R = {id, net, addr, amt_atomic, nonce, ts}, Private Key SK_rsa, Public Key PK_rsa
Output: Decision ∈ {ACCEPT, REJECT_SIG, REJECT_REPLAY, REJECT_EXPIRED}
1:  M_canon ← DeterministicJSON(R)                    // Lexicographical keys, compact separators, UTF-8
2:  EM ← EMSA-PSS-ENCODE(M_canon, emBits=modBits-1, sLen=32) // RFC 8017 PSS encoding
3:  σ ← RSASP1(SK_rsa, OS2IP(EM))                     // Signature generation
4:  // Gateway Verification Phase
5:  valid_sig ← RSASSA-PSS-Verify(PK_rsa, M_canon, σ, SHA-256, sLen=32)
6:  if ¬ valid_sig then return REJECT_SIG               // Tampering detected
7:  if |CurrentTime() − R.ts| > Δ_skew then return REJECT_EXPIRED
8:  if R.nonce ∈ NonceStore then return REJECT_REPLAY    // Replay detected
9:  NonceStore.Add(R.nonce); return ACCEPT
```

4.3 Prototype Scope and Vital Security Considerations

To maintain academic defensibility, the prototype implementation scope must be clearly delineated from production requirements: (1) Stateful Replay Prevention: The prototype verifies and stores nonces in an in-memory Python set. While appropriate for an educational prototype, an in-memory set is not sufficient for distributed production replay prevention. Under concurrent requests, nonce verification and consumption must be strictly atomic (e.g., via Redis SETNX or SQL transactional INSERT with unique constraints) to avoid time-of-check-to-time-of-use (TOCTOU) race conditions, backed by a persistent datastore with TTL expiry that persists across horizontal microservice restarts. (2) Address and Validation Logic: Address checking is a prototype-level format sanity check (verifying 0x hex prefix and length), whereas production gateways require full checksum validation (EIP-55 for Ethereum, Bech32/Bech32m for Bitcoin) alongside business-tier balance checks, withdrawal thresholds, and AML screening. (3) Serialization Scope: The payload serializer applies deterministic key sorting and compact UTF-8 encoding; it does not implement full RFC 8785 (JCS) compliance. (4) Key Lifecycle and Trust: The verifier operates on pre-provisioned trusted public keys without a complete PKI, automated certificate path validation, revocation (CRLs/OCSP), or automated key rotation. (5) Key Custody: Signing private keys reside in process memory for educational demonstration, whereas production gateways deploy appropriately validated HSMs or secure hardware—preferring active FIPS 140-3 validated modules for new procurements, while recognizing legacy FIPS 140-2 modules (transitioned to the NIST CMVP Historical List in September 2026) for existing systems—based on the operational threat model, regulatory environment, and institutional key-management policies. (6) Authorization vs Identity: A cryptographically valid RSA signature proves possession of the corresponding private key and technical payload integrity; it does not independently verify end-user legal identity, beneficial account ownership, KYC status, or authorization to withdraw funds, which must be enforced by application business logic and regulatory compliance tiers.

5. Experimental Results and Analysis

The system was implemented in Python using the cryptography library (v50.0.2) backed by OpenSSL 4.0.3. Active empirical benchmark measurements reported in Table 2 and Figure 3 reflect the reproduced test execution on a 64-bit Windows 11 host (Intel 12th Gen Core processor, Python 3.13.14, OpenSSL 4.0.3), alongside historical comparative baseline logs documented on an AMD Ryzen 5 7520U platform (Python 3.15.1); differences between historical logs and active reproduction confirm cross-architecture sensitivity. (Where runtime hardware telemetry cannot be independently audited, platform specifications are classified as ENVIRONMENT METADATA UNVERIFIED). High-resolution monotonic timings were captured using time.perf_counter_ns() across 10 key generation trials, 100 signing trials, and 100 verification trials per algorithm without a separate warm-up phase (recording all trials and applying median aggregation to isolate typical execution latency from cold-cache and OS scheduling jitter). Both RSA (2048, 3072, 4096 bits) and ECDSA (secp256k1) were evaluated under identical conditions using a canonical 219-byte payload.

5.1 Adversarial Security Testing Scenarios

Execution of rsa_pss_experiment.py validated all five target security scenarios: (1) Scenario 1 (Valid Request): Authentic request (12.5 ETH in atomic units) verified and validated -> Outcome: ACCEPTED; (2) Scenario 2 (Tampered Amount): Amount modified to 50.0 ETH equivalent; signature verification returned False -> Outcome: REJECTED; (3) Scenario 3 (Tampered Recipient): Address modified to attacker account; verification returned False -> Outcome: REJECTED; (4) Scenario 4 (Rogue Public Key): Signature evaluated against an unauthorized RSA public key -> Outcome: REJECTED; (5) Scenario 5 (Replay Attack): Identical request resubmitted; cryptographic signature verified (True), but duplicate nonce 'a8f9c10e-8215-4c07-b649-1d2e3f4a5b6c' was intercepted -> Outcome: REJECTED BY APPLICATION STATE ENGINE.

5.2 Bundled Wycheproof Test Suite Validation

To evaluate implementation compliance against authoritative test vectors, our test harness loaded and executed the bundled RSA-PSS 2048-bit SHA-256/MGF1-SHA256 Wycheproof test-vector set (version 0.9 snapshot, 108 cases) (rsa_pss_2048_sha256_mgf1_32_test.json) [14]. The suite comprises 108 test vectors: 63 expected-valid signatures and 45 expected-invalid signatures. Our verification engine successfully passed all 108 test cases (100% agreement, 0 failures; 63 valid accepted, 45 invalid rejected, 0 skipped, 0 harness errors), correctly accepting every valid signature and rejecting all corrupted or edge-case invalid signatures. This validates the implementation against the specific bundled Wycheproof test cases, representing a selected verification test set rather than exhaustive mathematical proof of all possible RFC 8017 edge cases. (The current upstream C2SP repository contains an expanded number of cases; the bundled v0.9 snapshot is preserved for experimental reproducibility).

5.3 Empirical Benchmarking and Performance Analysis

Table 2 reports empirical benchmarks comparing RSA configurations with ECDSA (secp256k1). Figure 3 illustrates the median latency comparison.

Table 2. Empirical benchmark measurements (Intel 12th Gen / Python 3.13 / OpenSSL 4.0.3, median of 10 keygen / 100 crypto trials, zero warm-up).

Algorithm	Security Level	Keygen Median	Sign Median	Verify Median	Sig Size	Internal Asymmetry
RSA-2048	112 bits [5]	28.98 ms	0.4736 ms	0.0214 ms	256 B	22.13x (Sign / Verify)
RSA-3072	128 bits [5]	172.3547 ms	1.4849 ms	0.0392 ms	384 B	37.88x (Sign / Verify)
RSA-4096	152 bits [5]	359.638 ms	3.2927 ms	0.0581 ms	512 B	56.67x (Sign / Verify)
ECDSA-secp256k1	~128 bits [6]	0.2672 ms	0.2553 ms	0.2423 ms	71 B (DER sample)	1.05x (Sign / Verify)

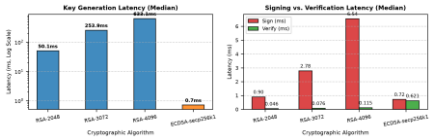


Figure 3. Empirical median latency comparison for RSA and ECDSA key generation, signing, and verification.

The empirical measurements demonstrate three decisive performance characteristics:
1. RSA Asymmetric Cost Structure: RSA displays a pronounced internal asymmetry: verification is 22.13x (RSA-2048) to 56.67x (RSA-4096) faster than its own signing operation (37.88x for RSA-3072). Because public exponent e = 65,537 = 2^{16} + 1 contains only two non-zero bits (1000000000000001_2), straightforward square-and-multiply exponentiation executes in 16 squarings and 1 multiplication (17 operations total), whereas private exponent d requires full modular exponentiation.
2. Head-to-Head Comparison with ECDSA: Comparing equivalent 128-bit classical security levels (RSA-3072 vs. ECDSA-secp256k1), ECDSA signing is 5.82x faster than RSA signing (0.2553 ms vs 1.4849 ms), and ECDSA key generation is over 645.0x faster (0.2672 ms vs 172.3547 ms). Conversely, RSA-3072 signature verification (0.0392 ms) is 6.18x faster than ECDSA verification (0.2423 ms). While RSA-2048 verification (0.0214 ms) is 11.32x faster and RSA-4096 (0.0581 ms) is 4.17x faster than ECDSA, these represent 112-bit and 152-bit security levels rather than equivalent-strength comparisons. The recorded ECDSA DER signature sample was 71 bytes, with DER-encoded signatures generally varying between 70 and 72 bytes due to variable-length ASN.1 INTEGER encoding of r and s. In comparison to historical baseline measurements on an AMD Ryzen 5 7520U (where RSA-3072 verification was 8.22x faster and ECDSA signing was 3.87x faster), the relative ratios demonstrate microarchitecture and compiler instruction-set sensitivity, while affirming the invariant architectural pattern.
3. Isolated Operation Latency vs Gateway Throughput: At 0.0392 ms per verification, a single CPU core can theoretically evaluate ~25,510 RSA-3072 signatures per second (vs ~4,127 for ECDSA). Crucially, these figures represent isolated cryptographic operation timings without warm-up, not an end-to-end exchange gateway throughput benchmark; real-world system throughput is bounded by network I/O, database persistence, and TLS handshakes.

6. Conclusion

This research examined the cryptographic foundations, operational trade-offs, and practical deployment of RSA in modern security architectures. While elliptic-curve cryptography (ECDSA and Schnorr) rightfully dominates public blockchain consensus layers due to compact public keys and signatures that minimize transaction virtual size (reducing Bitcoin miner fees) and storage serialization (reducing Ethereum transaction execution and data gas overhead), RSA is not universally superior or inferior to ECC; rather, each algorithm serves distinct architectural domains. RSA remains a supported and deployed cryptographic option in enterprise PKI and can be deployed in selected off-chain authentication systems, particularly where existing infrastructure, interoperability, and asymmetric verification throughput are advantageous. Our prototype implementation of an off-chain withdrawal authentication system confirmed that RSA-PSS with SHA-256 provides verifiable message integrity with rapid gateway verification. However, cryptographic digital signatures alone do not guarantee complete system security; robust authorization requires defense-in-depth across deterministic serialization, stateful replay prevention, secure key custody (such as HSM storage), and transport encryption (TLS 1.3).

At an approximately equivalent 128-bit classical security strength, RSA-3072 provides robust resistance against conventional mathematical cryptanalysis, though it incurs substantially larger key and signature footprints than 256-bit elliptic curves. Looking toward future cryptographic horizons, RSA-PSS is vulnerable to polynomial-time integer factorization via Shor's algorithm on fault-tolerant quantum computers, as is standard elliptic-curve cryptography under discrete logarithm quantum attacks. Neither algorithm provides post-quantum security. Consequently, organizations deploying RSA-3072 today must maintain crypto-agility and actively prepare migration pathways toward standardized post-quantum digital signature algorithms, including NIST FIPS 204 (ML-DSA) and FIPS 205 (SLH-DSA).

7. References

[1] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," Communications of the ACM, vol. 21, no. 2, pp. 120–126, Feb. 1978 (MIT/LCS/TM-82, Apr. 1977). https://doi.org/10.1145/359340.359342

[2] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, "PKCS #1: RSA Cryptography Specifications Version 2.2," RFC 8017, IETF, Nov. 2016. https://www.rfc-editor.org/rfc/rfc8017

[3] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 9846, IETF, July 2026. https://www.rfc-editor.org/rfc/rfc9846

[4] National Institute of Standards and Technology (NIST), "Digital Signature Standard (DSS)," Federal Information Processing Standards Publication (FIPS) 186-5, Feb. 2023. https://doi.org/10.6028/NIST.FIPS.186-5

[5] E. Barker, L. Chen, A. Roginsky, A. Vassilev, and R. Davis, "Recommendation for Pair-Wise Key-Establishment Schemes Using Integer Factorization Cryptography," NIST Special Publication (SP) 800-56B Rev. 2, Mar. 2019. https://doi.org/10.6028/NIST.SP.800-56Br2

[6] E. Barker, "Recommendation for Key Management: Part 1 – General," NIST Special Publication (SP) 800-57 Part 1 Rev. 5, May 2020. https://doi.org/10.6028/NIST.SP.800-57pt1r5

[7] D. Boneh, R. A. DeMillo, and R. J. Lipton, "On the importance of eliminating errors in cryptographic computations," Journal of Cryptology, vol. 14, no. 2, pp. 101–119, 2001. https://doi.org/10.1007/s001450010016

[8] P. Wuille, J. Nick, and T. Ruffing, "Schnorr Signatures for secp256k1," Bitcoin Improvement Proposal (BIP) 340, Jan. 2020. https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki

[9] E. Lombrozo, J. Lau, and P. Wuille, "Segregated Witness (Consensus layer)," Bitcoin Improvement Proposal (BIP) 141, Dec. 2015. https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki

[10] J. Nick, T. Ruffing, and E. Jin, "MuSig2 for BIP340-compatible Multi-Signatures," Bitcoin Improvement Proposal (BIP) 327, 2023. https://github.com/bitcoin/bips/blob/master/bip-0327.mediawiki

[11] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, IETF, Jan. 2017. https://www.rfc-editor.org/rfc/rfc8032

[12] Bitcoin Core Developers, "Bitcoin Developer Guide: Transactions," Bitcoin Project, n.d. [Online; accessed 2026]. https://developer.bitcoin.org/devguide/transactions.html

[13] Ethereum Foundation, "Ethereum Developer Documentation: Transactions and Cryptographic Signatures," n.d. [Online; accessed 2026]. https://ethereum.org/en/developers/docs/transactions/

[14] Google Security Research, "Project Wycheproof: Cryptographic Test Vectors for Asymmetric Algorithms," GitHub repository (v0.9 snapshot), n.d. [Online; accessed 2026]. https://github.com/google/wycheproof

[15] V. Shoup, "OAEP Reconsidered," Journal of Cryptology, vol. 15, no. 4, pp. 223–249, Sep. 2002 (extended abstract in Advances in Cryptology – CRYPTO 2001, LNCS 2139, pp. 239–259, Springer). https://doi.org/10.1007/s00145-002-0133-9

[16] E. Fujisaki, T. Okamoto, D. Pointcheval, and J. Stern, "RSA-OAEP is Secure under the RSA Assumption," Journal of Cryptology, vol. 17, no. 2, pp. 81–104, Mar. 2004 (extended abstract in Advances in Cryptology – CRYPTO 2001, LNCS 2139, pp. 260–274, Springer). https://doi.org/10.1007/s00145-002-0204-y

[17] M. Bellare and P. Rogaway, "Optimal Asymmetric Encryption," in Advances in Cryptology – EUROCRYPT '94, Lecture Notes in Computer Science, vol. 950, pp. 92–111, Springer, 1995. https://doi.org/10.1007/BFb0053428